

UNIVERSIDADE FEDERAL / INSTITUIÇÃO DE
ENSINO SUPERIOR

DISCIPLINA DE SISTEMAS DISTRIBUÍDOS

Sistema Distribuído para Auxílio de Dados Hospitalares

Relatório Técnico e Descritivo de Implementação: Marcos 1 e 2

Autor: Eduardo

Contexto: Projeto Acadêmico Prático

Foco: Telemetria Hospitalar, APIs Assíncronas, Persistência Local e
Balanceamento de Carga Round Robin

23 de setembro de 2026

Sumário

1	Introdução e Visão Geral	2
2	Análise e Modelagem do Dataset Hospitalar	2
2.1	Origem e Caracterização do Arquivo	2
2.2	Camada de Mapeamento (Schema Mapping)	3
3	Marco 1 — Simulação e Servidor Base	3
3.1	Arquitetura e Fluxo de Informação	3
3.2	O Produtor de Telemetria (<code>client/csv_sender.py</code>)	3
3.3	O Servidor Base (<code>server/app.py</code>)	4
3.4	Camada de Persistência (<code>server/database.py</code>)	4
4	Marco 2 — Balanceamento de Carga e Múltiplas Instâncias	5
4.1	Arquitetura Distribuída Proposta	5
4.2	Algoritmo Round Robin (Formulação e Implementação)	5
4.3	Identificação e Transparência	5
4.4	Bancos de Dados Independentes no Marco 2	6
5	Decisões de Engenharia e Desafios Superados	6
6	Validação e Resultados Experimentais	7
7	Roteiro Prático de Demonstração e Verificação	7
7.1	Etapa Prévia: Ambiente e Testes Rápidos	7
7.2	Demonstração Prática do Marco 1 (Servidor Base)	7
7.3	Demonstração Prática do Marco 2 (Balanceamento de Carga)	8
7.4	Evidências de Funcionamento que Devem Ser Apontadas	9
8	Conclusão	9

1 Introdução e Visão Geral

O monitoramento contínuo de pacientes em ambientes hospitalares gera fluxos ininterruptos de telemetria crítica (sinais vitais, alertas fisiológicos e ocorrências de emergência). Em arquiteturas computacionais centralizadas, esse volume de requisições pode sobrecarregar servidores de borda, introduzir latência no processamento e criar pontos únicos de falha (*Single Points of Failure*).

O presente projeto tem por finalidade o projeto e a implementação de um **Sistema Distribuído para Auxílio de Dados Hospitalares**, concebido com propósitos estritamente acadêmicos e didáticos. A proposta adota uma abordagem incremental estruturada em marcos evolutivos. Este documento detalha a concepção, formalização e validação dos dois primeiros marcos:

- **Marco 1 — Simulação e Servidor Base:** Ingestão progressiva de telemetria real proveniente de um conjunto de dados hospitalar (*dataset* Kaggle), validação estrita de esquemas em camada de API RESTful (FastAPI), conversão estruturada de tipos e persistência relacional local (SQLite).
- **Marco 2 — Balanceamento de Carga e Múltiplas Instâncias:** Criação de três instâncias concorrentes do servidor e de um Balanceador de Carga em camada de aplicação implementando o algoritmo *Round Robin*, com transparência de acesso e identificação inequívoca de nós processadores.

O projeto foi integralmente construído em linguagem Python (versão 3.10+), priorizando clareza conceitual, desacoplamento e portabilidade.

2 Análise e Modelagem do Dataset Hospitalar

2.1 Origem e Caracterização do Arquivo

A fonte de dados empregada é o arquivo CSV obtido via Kaggle:

```
Synthetic_patient-HealthCare-Monitoring_dataset.csv
```

Diferente de geradores sintéticos aleatórios gerados em tempo de execução, este arquivo atua como repositório de telemetria pré-coletada.

Uma análise exploratória minuciosa do arquivo revelou:

- **Volume Total:** 60.000 registros clínicos tabulados (60.001 linhas contando o cabeçalho).
- **Completude dos Dados:** Não foram identificados registros nulos ou incompletos (0% de valores ausentes em todas as colunas).
- **Ausência de Carimbo de Tempo:** O conjunto original não possui coluna temporal (*timestamp*), exigindo que a camada de borda do servidor gere um carimbo no momento exato do recebimento da mensagem.

2.2 Camada de Mapeamento (Schema Mapping)

Para adequar a terminologia do arquivo (em língua inglesa e com caracteres especiais) às boas práticas de Engenharia de Software e APIs RESTful em português, foi implementado um mapeamento bidirecional no módulo `server/models.py`.

Tabela 1: Mapeamento de Atributos do CSV Kaggle para a API e Banco de Dados

Coluna Original (CSV)	Campo na API/DB	Tipo	Descrição Semântica
Patient Number	<code>paciente_id</code>	<code>int</code>	Identificador do paciente
Heart Rate (bpm)	<code>frequencia_cardiaca</code>	<code>int</code>	Frequência cardíaca (bpm)
SpO2 Level (%)	<code>spo2</code>	<code>int</code>	Saturação de oxigênio
Systolic Blood Pressure	<code>pressao_sistolica</code>	<code>int</code>	Pressão sistólica (mmHg)
Diastolic Blood Pressure	<code>pressao_diastolica</code>	<code>int</code>	Pressão diastólica (mmHg)
Body Temperature (°C)	<code>temperatura</code>	<code>float</code>	Temperatura corpórea (°C)
Fall Detection	<code>deteccao_queda</code>	<code>str</code>	Indicador de queda (Yes/No)
Predicted Disease	<code>doenca_prevista</code>	<code>str</code>	Diagnóstico clínico previsto
Data Accuracy (%)	<code>precisao_dados</code>	<code>int</code>	Confiabilidade do sensor (%)
Heart Rate Alert	<code>alerta_freq_cardiaca</code>	<code>str</code>	Alerta de FC (NORMAL/ABNORMAL)
SpO2 Level Alert	<code>alerta_spo2</code>	<code>str</code>	Alerta de SpO2
Blood Pressure Alert	<code>alerta_pressao</code>	<code>str</code>	Alerta de pressão arterial
Temperature Alert	<code>alerta_temperatura</code>	<code>str</code>	Alerta de temperatura

3 Marco 1 — Simulação e Servidor Base

3.1 Arquitetura e Fluxo de Informação

O Marco 1 estabelece o pipeline elementar de telemetria hospitalar unidirecional. A Figura 1 ilustra os componentes e o ciclo de vida do dado.



Figura 1: Fluxo de telemetria do Marco 1: do arquivo tabular à persistência local.

3.2 O Produtor de Telemetria (`client/csv_sender.py`)

O script atua como um emulador de dispositivos biomédicos de leito. Suas responsabilidades incluem:

1. Abertura segura do arquivo CSV via `csv.DictReader`;
2. Conversão atômica de cada linha através da função `converter_registro_csv`, efetuando o *casting* de strings para inteiros e números de ponto flutuante;
3. Envio progressivo via requisição síncrona HTTP POST com corpo em formato JSON para o endpoint de destino;

4. Temporização configurável entre eventos (`time.sleep(intervalo)`), viabilizando simulações aceleradas ou em tempo real;
5. Tratamento resiliente de falhas de conexão (`ConnectionError` e `Timeout`), garantindo que falhas momentâneas de rede não interrompam o processo.

3.3 O Servidor Base (server/app.py)

Construído sobre o framework assíncrono **FastAPI**, o servidor expõe três interfaces essenciais:

- **POST /dados:** Recebe a carga útil validada pelo modelo `DadosPaciente` (Pydantic). O servidor injeta o `timestamp` da máquina e seu identificador (`server_id`), grava no banco e retorna HTTP 200 com a entidade persistida.
- **GET /dados:** Retorna registros armazenados, oferecendo suporte nativo à paginação através dos parâmetros de consulta `limit` e `offset` (essencial para evitar estouro de memória em datasets com dezenas de milhares de linhas).
- **GET /dados/{paciente_id}:** Filtra o histórico longitudinal de um paciente específico, retornando HTTP 404 caso o identificador não tenha medições registradas.

3.4 Camada de Persistência (server/database.py)

Utiliza o mecanismo **SQLite3**. Cada instância do servidor gerencia seu próprio arquivo de banco de dados, inicializando a tabela `dados_pacientes` na inicialização:

```
1 CREATE TABLE IF NOT EXISTS dados_pacientes (  
2     id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     paciente_id INTEGER NOT NULL,  
4     frequencia_cardiaca INTEGER NOT NULL,  
5     spo2 INTEGER NOT NULL,  
6     pressao_sistolica INTEGER NOT NULL,  
7     pressao_diastolica INTEGER NOT NULL,  
8     temperatura REAL NOT NULL,  
9     deteccao_queda TEXT NOT NULL,  
10    doenca_prevista TEXT NOT NULL,  
11    precisao_dados INTEGER NOT NULL,  
12    alerta_freq_cardiaca TEXT NOT NULL,  
13    alerta_spo2 TEXT NOT NULL,  
14    alerta_pressao TEXT NOT NULL,  
15    alerta_temperatura TEXT NOT NULL,  
16    timestamp TEXT NOT NULL,  
17    server_id TEXT NOT NULL  
18 );
```

Listing 1: Estrutura da Tabela no SQLite

4 Marco 2 — Balanceamento de Carga e Múltiplas Instâncias

4.1 Arquitetura Distribuída Proposta

No Marco 2, a arquitetura é escalada horizontalmente. Introduce-se um nó intermediário (o Balanceador de Carga) na porta 8000, responsável por desacoplar o cliente produtor dos servidores de aplicação (instanciados nas portas 8001, 8002 e 8003).

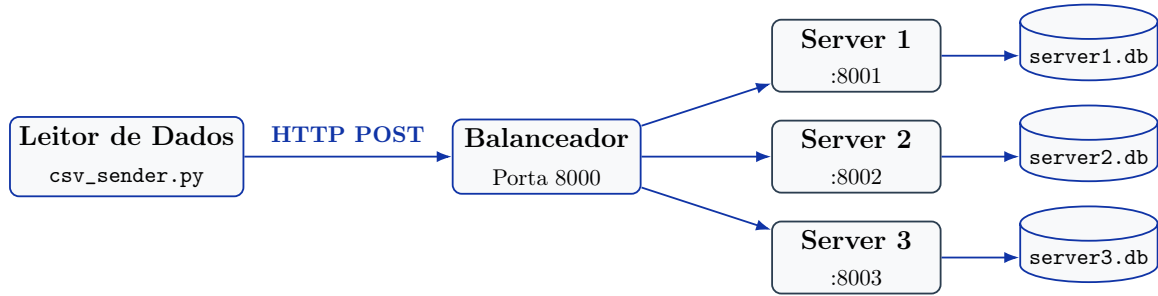


Figura 2: Topologia física e lógica do Marco 2: balanceamento em anel uniforme.

4.2 Algoritmo Round Robin (Formulação e Implementação)

O algoritmo **Round Robin** (revezamento circular) distribui as tarefas sequencialmente ao longo de uma lista ordenada de nós disponíveis. Dado um conjunto estático de N servidores $\mathcal{S} = \{s_0, s_1, \dots, s_{N-1}\}$ e um contador de requisições acumuladas $r \in \mathbb{N}$ (iniciado em 0):

$$\text{Servidor Alvo}(r) = s_{r \pmod{N}} \quad (1)$$

Esta regra garante distribuição perfeitamente equitativa da carga útil de requisições:

- Requisição 1 ($r = 0$): Servidor 0 (**server-01** na porta 8001)
- Requisição 2 ($r = 1$): Servidor 1 (**server-02** na porta 8002)
- Requisição 3 ($r = 2$): Servidor 2 (**server-03** na porta 8003)
- Requisição 4 ($r = 3$): Servidor 0 (**server-01** na porta 8001)

O balanceador foi implementado através da classe `RoundRobinBalancer`, no módulo `load_balancer/balancer.py`. Para o encaminhamento assíncrono das requisições, foi empregada a biblioteca `httpx.AsyncClient`.

4.3 Identificação e Transparência

Cada instância do servidor executa o mesmo código-fonte, porém é parametrizada via linha de comando com identidade e banco próprios:

- **Instância 1:** Porta 8001, ID "**server-01**", banco `databases/server1.db`
- **Instância 2:** Porta 8002, ID "**server-02**", banco `databases/server2.db`

- **Instância 3:** Porta 8003, ID "server-03", banco `databases/server3.db`

Quando o balanceador despacha a requisição e recebe a resposta do nó selecionado, a resposta HTTP 200 propagada de volta ao cliente contém explicitamente o campo:

```
"server_id": "server-01"
```

Isso permite auditoria em tempo real pelo cliente da distribuição de carga sem quebrar a *transparência de localização* (o cliente continua se comunicando unicamente com o endereço `http://localhost:8000`).

4.4 Bancos de Dados Independentes no Marco 2

Conforme estipulado no planejamento pedagógico do projeto, **nenhuma replicação ou sincronização entre bancos foi implementada neste estágio**.

Cada servidor alimenta estritamente seu próprio banco SQLite local (`server1.db`, `server2.db` ou `server3.db`). Se 12 registros forem enviados pelo balanceador, cada um dos 3 bancos registrará precisamente 4 tuplas. A persistência distribuída consistente e a replicação de réplicas ativas/passivas constituem o escopo do futuro Marco 3.

5 Decisões de Engenharia e Desafios Superados

Durante o desenvolvimento dos Marcos 1 e 2, algumas decisões de engenharia de software foram determinantes para a robustez da solução:

1. **Substituição de Manipuladores Depreciados do FastAPI:** Versões recentes do FastAPI descontinuaram o padrão decorador `@app.on_event("startup")`. Adotou-se o gerenciador de contexto de ciclo de vida assíncrono (`lifespan`), combinado com a invocação antecipada e idempotente de `init_db()`, permitindo que os testes com `TestClient` operassem sem falhas de tabela inexistente.
2. **Preservação da Integridade do Dataset Original:** O arquivo CSV de 4,4 MB não foi duplicado nem modificado. Criou-se um *symbolic link* (`data/dataset.csv`) referenciando o original, garantindo reproducibilidade sem redundância de armazenamento.
3. **Isolamento de Concorrência nos Testes do Marco 2:** Em testes automatizados envolvendo servidores reais e balanceadores em segundo plano, a reutilização rápida de portas TCP pode causar o erro `Errno 98: Address already in use`. A solução adotada consistiu em empregar *fixtures* de escopo de classe (`scope="class"`) no `pytest` com portas dinâmicas na faixa 19000, eliminando condições de corrida e travamentos em sockets.

6 Validação e Resultados Experimentais

O sistema foi rigorosamente validado através de 26 testes automatizados construídos com `pytest`, subdivididos em duas suítes dedicadas.

Tabela 2: Sumário da Suíte de Testes Automatizados

Módulo de Teste	Qtd.	Objetivo da Validação
tests/test_marco1.py	17	Existência do CSV, presença de colunas, parsing sem perda, validação Pydantic, inserção no SQLite, queries com paginação (<code>limit/offset</code>) e busca por paciente.
tests/test_marco2.py	9	Aritmética modular do Round Robin, inicialização simultânea de 3 servidores e do balanceador, roteamento HTTP transparente e partição homogênea nos bancos.
Total de Testes Aprovados	26/26	Taxa de Sucesso: 100% (tempo de execução: 0,92s)

7 Roteiro Prático de Demonstração e Verificação

Para apresentar o projeto ao professor ou avaliadores e comprovar inequivocamente que os Marcos 1 e 2 estão plenamente funcionais, deve-se seguir o procedimento estruturado a seguir.

7.1 Etapa Prévia: Ambiente e Testes Rápidos

No terminal principal, ative o ambiente virtual e execute a suíte de testes:

```
1 cd "Projeto SD"
2 source venv/bin/activate
3 python -m pytest tests/ -v
```

O que mostrar: A saída verde confirmando que todos os 26 testes passaram com sucesso em menos de 1 segundo.

7.2 Demonstração Prática do Marco 1 (Servidor Base)

Para demonstrar o funcionamento elementar do Marco 1:

1. Terminal 1 (Servidor):

```
1 python server/app.py --port 8001 --server-id server-01 --db-path databases/server1.db
```

2. Terminal 2 (Envio Direto):

```
1 python client/csv_sender.py --url http://localhost:8001/dados --intervalo 0.3 --limite 5
```

3. O que explicar ao avaliador:

- No Terminal 2, aponte que os registros 1 a 5 foram lidos do CSV real e enviados via HTTP POST.
- No Terminal 1, mostre os logs registrando o recebimento dos sinais vitais.

- Abra o navegador ou utilize o `curl` para demonstrar a consulta dos dados armazenados:

```
1      curl -s http://localhost:8001/dados | python -m json.  
      tool
```

- Consulte o banco SQLite diretamente pelo terminal para provar a persistência:

```
1      sqlite3 databases/server1.db "SELECT id, paciente_id,  
      frequencia_cardiaca, timestamp, server_id FROM  
      dados_pacientes;"
```

7.3 Demonstração Prática do Marco 2 (Balanceamento de Carga)

Esta é a comprovação máxima do sistema distribuído. Feche a execução do teste anterior e prepare os 5 terminais:

- **Terminal 1 (Server 1):**

```
1      python server/app.py --port 8001 --server-id server-01 --db-  
      path databases/server1.db
```

- **Terminal 2 (Server 2):**

```
1      python server/app.py --port 8002 --server-id server-02 --db-  
      path databases/server2.db
```

- **Terminal 3 (Server 3):**

```
1      python server/app.py --port 8003 --server-id server-03 --db-  
      path databases/server3.db
```

- **Terminal 4 (Balanceador):**

```
1      python load_balancer/balancer.py
```

- **Terminal 5 (Cliente Produtor):** Envie um lote controlado de 12 requisições através do balanceador (porta 8000):

```
1      python client/csv_sender.py --url http://localhost:8000/dados  
      --intervalo 0.4 --limite 12
```

7.4 Evidências de Funcionamento que Devem Ser Apontadas

1. **Alternância Visual no Balanceador (Terminal 4):** O balanceador exibirá explicitamente o revezamento cíclico das 12 requisições:

```
1 [BALANCEADOR] Requisicao 1 -> http://localhost:8001  
2 [BALANCEADOR] Requisicao 2 -> http://localhost:8002  
3 [BALANCEADOR] Requisicao 3 -> http://localhost:8003  
4 [BALANCEADOR] Requisicao 4 -> http://localhost:8001  
5 [BALANCEADOR] Requisicao 5 -> http://localhost:8002  
6 ...
```

2. **Reconhecimento pelo Cliente (Terminal 5):** O leitor confirmará o processamento indicando qual servidor respondeu:

```

1 [ 1] Paciente      1 -> server-01 | FC=98  SpO2=96 PA=120/86 Temp=38.1
2 [ 2] Paciente      2 -> server-02 | FC=105 SpO2=97 PA=177/104 Temp=37.6
3 [ 3] Paciente      3 -> server-03 | FC=90   SpO2=85 PA=139/57 Temp=37.0
4 [ 4] Paciente      4 -> server-01 | FC=102 SpO2=87 PA=101/77 Temp=36.4

```

3. **Divisão Exata nos Três Bancos SQLite:** Abra um terminal adicional e execute a contagem em cada banco:

```

1 echo "--- Server 1 (8001) ---"
2 sqlite3 databases/server1.db "SELECT COUNT(*) FROM
   dados_pacientes;"
3 echo "--- Server 2 (8002) ---"
4 sqlite3 databases/server2.db "SELECT COUNT(*) FROM
   dados_pacientes;"
5 echo "--- Server 3 (8003) ---"
6 sqlite3 databases/server3.db "SELECT COUNT(*) FROM
   dados_pacientes;"

```

Resultado Incontestável: Cada um dos três bancos exibirá exatamente **4 registros** (totalizando os 12 enviados). Isso prova que a carga foi perfeitamente distribuída e que as instâncias operam com persistência isolada.

8 Conclusão

Os objetivos previstos para os Marcos 1 e 2 foram plenamente alcançados. A solução atende a todos os requisitos didáticos de Sistemas Distribuídos:

- Demonstração de heterogeneidade e acoplamento fraco mediante o protocolo HTTP e JSON;
- Preservação de dados reais de telemetria sem artifícios sintéticos desnecessários;
- Implementação de balanceamento de carga Round Robin com transparência de acesso e tolerância a falhas na camada cliente;
- Estruturação modular que viabiliza a evolução suave para o Marco 3 (sincronização e replicação de dados distribuídos).